

Laboratory of Mobile Operating Systems

Creating a basic Android service

Introduction

Services are special components used for long-running background tasks, such as maintaining connections, playing sounds and handling inter-process communication. A simple service capable of running background jobs can look like this:

```
class MyService : LifecycleService() {
    private val binder = LocalBinder()

    inner class LocalBinder : Binder() {
        fun getService(): MyService = this@MyService
    }

    override fun onBind(intent: Intent): IBinder {
        super.onBind(intent)
        return binder
    }

    override fun onCreate() {
        super.onCreate()
        // Start long running jobs here.
    }

    override fun onDestroy() {
        super.onDestroy()
        // Cancel any remaining jobs here.
    }
}
```

Naturally, the *lifecycleScope* methods require adding the following dependency to the app module's build.gradle file:

```
implementation ("androidx.lifecycle:lifecycle-runtime-ktx:2.7.0")
```

Important! Your service won't run unless it is declared in the *AndroidManifest.xml* file:

```
<service
    android:name=".MyService"
    android:enabled="true"
    android:exported="false" />
```

There are two ways a client component can bring a service to life. The first way is to simply start the service – it will keep running until it either calls the **stopSelf()** method, or the starting component calls the **stopService()** method:

```
val intent = Intent(this, MyService::class.java)
startService(intent)
```

The second way is to bind to the service as a client. Using this method, the service should stay alive until all the clients call the **unbindService()** method:

```
val intent = Intent(this, MyService::class.java)
bindService(intent, serviceConnection, Context.BIND_AUTO_CREATE)
```

Binding to a service requires defining additional variables in the client component. Using the following code, the client application will be able to access the *mService* methods and variables as long as *mBound* is equal to true.

```
private lateinit var mService: MyService
private var mBound: Boolean = false

private val serviceConnection = object : ServiceConnection {
    override fun onServiceConnected(className: ComponentName,
                                     service: IBinder) {
        val binder = service as MyService.LocalBinder
        mService = binder.getService()
        mBound = true
    }
    override fun onServiceDisconnected(arg0: ComponentName) {
        mBound = false
    }
}
```

Finally, the service can be both **Started** and **Bound**. This way, the service is unlikely to be destroyed until one of the stop methods is called and all of its clients perform the unbinding:

```
val intent = Intent(this, MyService::class.java)
startService(intent)
bindService(intent, serviceConnection, Context.BIND_AUTO_CREATE)
```

Task 1. (2 points)

Create a simple service that will keep counting the amount of time passed between its creation and its destruction.

Task 2. (1 point)

Create an activity capable of binding to your service and unbinding from it. The binding and unbinding can happen either by the user interface, or the activity's proper lifecycle methods, such as *onStart()* and *onStop()*.

Task 3. (1 point)

In your activity, keep displaying the service's running time on a regular basis.

Variant I

The service should keep informing the activity about the updated running time. The

simplest way of achieving this might be by defining a new interface, making your activity implement it, and sending your activity object to the service:

```
// Interface defined inside the service:
interface ServiceCallback {
    fun onGetResult(intResult: Int)
}
// Reference to your activity, with a function for obtaining it:
private var callback: ServiceCallback? = null
fun obtainCallback(callback: ServiceCallback?) {
    this.callback = callback
}
var runningTime: Int = 0 // update this from time to time
// You can call the activity method like this:
// callback?.onGetResult(runningTime)

// Make your activity implement the interface:
class MainActivity : AppCompatActivity(),
    MyService.ServiceCallback {
    override fun onGetResult(intResult: Int) {
        // Get the elapsed time here.
    }
}
// Call this function after binding to the service:
// mService.obtainCallback(this@MainActivity)
```

Variant II

The activity should regularly ask the service about its running time. This can be achieved by accessing the service's methods or variables inside the activity.

Task 4. (1 point)

Add a foreground notification to your service. First, you will need to update your *AndroidManifest.xml* with information regarding the type of foreground service type:

```
<uses-permission
    android:name="android.permission.FOREGROUND_SERVICE"/>
<uses-permission
    android:name="android.permission.FOREGROUND_SERVICE_DATA_SYNC"/>
<uses-permission
    android:name="android.permission.POST_NOTIFICATIONS"/>
```

as well as the foreground service type:

```
<service
    ...
    android:foregroundServiceType="dataSync" />
```

If your app is launched on Android 13 or later, it will need to ask the user for a proper permission during runtime...

```
if (ContextCompat.checkSelfPermission(this, POST_NOTIFICATIONS)
    == PackageManager.PERMISSION_DENIED
    && Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
    ActivityCompat.requestPermissions(this,
```

```
    arrayOf(POST_NOTIFICATIONS), 0)
}
```

and verify if this permission was actually granted:

```
override fun onRequestPermissionsResult(requestCode: Int,
permissions: Array<String>, grantResults: IntArray) {
    super.onRequestPermissionsResult(requestCode, permissions,
grantResults)
    if (requestCode == 0) {
        if (grantResults.isNotEmpty() && grantResults[0] ==
PackageManager.PERMISSION_GRANTED) {
            Toast.makeText(this@MainActivity, "Notifications
Permission Granted", Toast.LENGTH_SHORT).show()
        } else {
            Toast.makeText(this@MainActivity, "Notifications
Permission Denied", Toast.LENGTH_SHORT).show()
        }
    }
}
```

Finally, create a new notification from your service:

```
val CHANNEL_ID = "my_channel_01"
val channel = NotificationChannel(CHANNEL_ID, "My Channel",
NotificationManager.IMPORTANCE_DEFAULT)
(getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager).
    createNotificationChannel(channel)
val notification = NotificationCompat.Builder(this,
CHANNEL_ID).build()
ServiceCompat.startForeground(this, 101, notification,
ServiceInfo.FOREGROUND_SERVICE_TYPE_DATA_SYNC)
```

Note: the above code may need some workarounds depending on the system version ;-)